

CAPSLOCK: ENVIRONMENT-AWARE POLICY ENGINE

*Automating Compliance-Aligned Kubernetes Security Policy
Enforcement Through Environment Classification*

Nethmi Kaavya Raigambandarage

(IT22338716)

Bachelor of Science (Hons) in Information Technology
Specialising in Cyber Security

Department of Computer Systems Engineering

Sri Lanka Institute of Information Technology
Sri Lanka

April 2026

DECLARATION

I declare that this is my own work and this dissertation does not incorporate without acknowledgement any material previously submitted for a Degree or Diploma in any other University or institute of higher learning and to the best of my knowledge and belief it does not contain any material previously published or written by another person except where the acknowledgement is made in the text.

Also, I hereby grant to Sri Lanka Institute of Information Technology, the non-exclusive right to reproduce and distribute my dissertation, in whole or in part in print, electronic or other medium. I retain the right to use this content in whole or part in future works (such as articles or books).

Signature: _____ Date: 26.04.2026

The above candidate has carried out research for the Bachelor's Degree Dissertation under my supervision.

Signature of Supervisor: _____ Date:

Name of Supervisor: _____

DEDICATION

To my family, for their unwavering support and encouragement throughout this research.

To the CAPSLock research team - Marlon, Themiya, and Senali - for the collaboration, patience, and shared purpose that made this work possible.

To Mr. Amila Senarathne and Mr. Kavinga Yapa Abeywardena, whose guidance turned a research problem into a working system — and whose patience made the process worthwhile.

ACKNOWLEDGEMENTS

The author wishes to express sincere gratitude to the supervisor, Mr. Amila Senarathne, and co-supervisor, Mr. Kavinga Yapa Abeywardena, at the Sri Lanka Institute of Information Technology for their continuous guidance, constructive criticism, and technical feedback throughout the duration of this research. Their expertise in cloud-native security and distributed systems was instrumental in shaping the design decisions underpinning the Environment-Aware Policy Engine.

Appreciation is extended to the members of the CAPSLock research group — Kulatunga K K M D, Pandukabhaya H. D. T. and S. A. L. Guruge — for their productive collaboration on integration contracts, joint system testing, and system-level validation. The cohesive nature of the CAPSLock platform was only achievable through this close coordination across all four component teams.

Gratitude is also due to the Department of Computer Systems Engineering at SLIIT, whose provision of laboratory infrastructure, access to academic databases, and supportive faculty made this research possible. The feedback received during progress reviews contributed significantly to the refinement of the research problem and evaluation methodology.

Finally, sincere appreciation is extended to family and friends for their patience, encouragement, and unwavering support throughout the writing of this thesis.

ABSTRACT

Modern Kubernetes deployments operate across multiple environment tiers — development, staging, and production — each carrying fundamentally different security and compliance requirements. Despite this, the dominant admission control frameworks, OPA Gatekeeper and Kyverno, enforce static, manually curated policy sets with no mechanism for automatic adaptation to deployment context. This forces engineering teams to maintain separate policy catalogues per tier, a practice that empirical research has consistently shown to be error-prone and operationally unsustainable.

This dissertation presents the Environment-Aware Policy Engine (EAPE), Component 2 of the CAPSLock Kubernetes security gateway. EAPE implements a seven-step automated policy application pipeline designed to replace manual, static policy management with context-driven, automated enforcement. The system is implemented in Go 1.21 and Python 3.11 via FastAPI, with an 83.6% unit test coverage baseline.

EAPE's environment detection uses a seven-factor weighted confidence scoring algorithm producing a detection result and a confidence value in $[0, 1]$ that allows the system to require human override below configurable thresholds. Conflict detection covers five dimensions and supports three resolution strategies. Compliance validation covers CIS Kubernetes Benchmark v1.9 (28 controls) and PCI-DSS v4.0 (16 requirements). Performance benchmarking confirms that policy selection completes in 882 ns/op, conflict detection in 7.2 μ s, and the complete pipeline in 163 ms per namespace. The system reduces manual policy configuration overhead by 73%.

Keywords: Kubernetes Security, Environment Classification, Policy-as-Code, OPA Gatekeeper, Kyverno, CIS Benchmarks, PCI-DSS, Compliance Automation, Go, FastAPI

TABLE OF CONTENTS

DECLARATION	2
DEDICATION.....	3
ACKNOWLEDGEMENTS.....	4
ABSTRACT.....	5
TABLE OF CONTENTS.....	6
LIST OF FIGURES	9
LIST OF TABLES.....	10
LIST OF ABBREVIATIONS.....	11
1. INTRODUCTION.....	12
1.1. Background and Context	12
1.2. Research Gap.....	13
1.3. Research Problem.....	14
1.4. Research Objectives	15
1.5. Thesis Structure	15
2. LITERATURE REVIEW.....	16
2.1. Kubernetes Security: Empirical Evidence of the Problem	16
2.2. Admission Control and Policy Enforcement Tools	16
2.3. Policy-as-Code and Compliance Automation	17
2.4. Environment Classification in Cloud Systems	17
2.5. Compliance Frameworks: CIS Benchmarks and PCI-DSS	18
2.6. Policy Conflict Detection and Resolution	18
2.7. Summary and Research Positioning	18
3. SYSTEM ARCHITECTURE.....	19
3.1. CAPSLock: The Broader Research System	19
3.2. EAPE's Role and Integration Points	20
3.3. EAPE High-Level Architecture.....	20
3.4. Technology Stack and Design Decisions	21
4. METHODOLOGY.....	22
4.1. Research Approach and Design Philosophy.....	22
4.2. Requirements Analysis.....	22
4.2.1. Functional Requirements.....	22
4.2.2. Non-Functional Requirements	23
4.2.3. Integration Requirements	24
4.3. Design of the Seven-Factor Detection Algorithm	24

4.3.1.	Algorithm Design Rationale.....	24
4.3.2.	Detection Factors and Weights	25
4.3.3.	Example Detection Outcomes	26
4.4.	The Seven-Step Policy Application Pipeline.....	27
4.4.1.	Step 1 — Environment Detection	27
4.4.2.	Step 2 — Policy Selection.....	27
4.4.3.	Step 3 — Conflict Detection	27
4.4.4.	Step 4 — Conflict Resolution	27
4.4.5.	Step 5 — Compliance Validation.....	27
4.4.6.	Step 6 — Status Reporting	28
4.4.7.	Step 7 — Service Verification	28
4.5.	Compliance Framework Integration Design.....	28
4.5.1.	CIS Benchmark v1.9 Integration.....	28
4.5.2.	PCI-DSS v4.0 Integration	29
4.6.	Conflict Detection and Resolution Design	29
4.6.1.	Five-Dimension Conflict Classification.....	29
4.6.2.	Three Resolution Strategies	30
4.6.3.	Conflict Audit Persistence.....	30
4.7.	Policy Template Architecture.....	30
4.7.1.	Three Environment Templates	30
4.7.2.	Policy Inheritance Design	31
4.8.	Advanced Feature Design.....	31
4.8.1.	Continuous Change Detection.....	31
4.8.2.	Local State Persistence.....	31
4.8.3.	CLI Interface	31
4.9.	Development Process and Iterative Refinement.....	32
4.9.1.	Development Phases	32
4.9.2.	Key Design Iterations.....	32
4.10.	Testing Strategy and Validation Design.....	33
4.10.1.	Unit Testing.....	33
4.10.2.	End-to-End Testing	33
4.10.3.	Integration Testing via httpptest.....	33
5.	IMPLEMENTATION	34
5.1.	Development Environment and Infrastructure	34
5.2.	Go Policy Engine Binary	34
5.2.1.	Environment Detector	34
5.2.2.	Policy Selector	34

5.2.3.	Conflict Detector and Resolver	34
5.2.4.	Compliance Validator	35
5.3.	Python FastAPI Integration Bridge	35
5.4.	Compliance Validation Packages	35
5.5.	Policy Template System	35
5.6.	Management Dashboard	36
5.7.	Integration with CAPSLock Components	38
5.7.1.	MEDS Integration	38
5.7.2.	SDLB Integration	38
5.7.3.	ICAP Operator Integration	38
6.	EVALUATION	39
6.1.	Experimental Setup	39
6.2.	Environment Detection Accuracy	39
6.2.1.	Detection Accuracy Results	39
6.2.2.	End-to-End Detection Test Results	40
6.3.	Conflict Detection and Resolution	40
6.4.	Compliance Coverage and Correctness	41
6.4.1.	Policy Selection and Translation Correctness	42
6.5.	Performance Benchmarks	42
6.6.	Test Coverage	43
6.7.	End-to-End and Integration Test Results	44
6.8.	Operational Overhead Reduction	45
6.9.	Comparison with Existing Approaches	45
7.	CONCLUSION	46
7.1.	Summary of Contributions	46
7.2.	Research Findings and Implications	47
7.3.	Limitations	48
7.4.	Future Work	48

LIST OF FIGURES

Figure 1: Policy Conflict Detector — No conflicts detected. Same configuration with CIS framework selected instead of PCI-DSS. Result: policy configuration is valid.	37
Figure 2: Policy Conflict Detector — Single compliance conflict. Permissive enforcement, privileged pod security standard, PCI-DSS framework, development environment. Result: 1 HIGH/COMPLIANCE conflict.	37
Figure 3: Policy Conflict Detector — Two compliance conflicts. PCI-DSS framework with network policies disabled. Result: 2 HIGH/COMPLIANCE conflicts.	37
Figure 4: Policy Conflict Detector — Mixed severity conflicts in production. Strict enforcement with high risk level. Result: 1 MEDIUM/ENFORCEMENT + 1 HIGH/COMPLIANCE conflict.	37

LIST OF TABLES

Table 1: CAPSLock Component Summary and EAPE Integration Points.....	19
Table 2: EAPE Internal Architecture — Packages and Responsibilities	21
Table 3: Detection Factors and Weights (Source: pkg/detector/environment_detector.go)	26
Table 4: Example Detection Outcomes (Source: Technical Validation Report, Section 4.2)	26
Table 5: CIS Benchmark v1.9 Control Distribution Across Sections.....	28
Table 6: PCI-DSS v4.0 Requirement Group Distribution	29
Table 7: Five Conflict Dimensions with Severity Ratings (Source: pkg/conflict/detector.go)	29
Table 8: EAPE Policy Tiers with Enforcement and ICAP Mode	30
Table 9: Environment Detection Results Across Test Namespaces	39
Table 10: End-to-End Detection and Policy Application Test Results.....	40
Table 11: Conflict Detection and Resolution Test Results	41
Table 12: CIS Benchmark v1.9 Compliance Results.....	42
Table 13: Policy Selection and Translation Correctness.....	42
Table 14: Performance Benchmark Results (Source: tests/performance/benchmark_test.go).....	43
Table 15: Unit Test Coverage by Package (Source: go test ./... -cover)	43
Table 16: End-to-End Test Results (All 6/6 Pass)	44
Table 17: Integration Test Results via httpstest (All 9/9 Pass).....	45

LIST OF ABBREVIATIONS

Abbreviation	Description
API	Application Programming Interface
CAPSLock	Container-Aware Policy Security Lock
CEL	Common Expression Language
CIS	Centre for Internet Security
CRD	Custom Resource Definition
EAPE	Environment-Aware Policy Engine
ICAP	Internet Content Adaptation Protocol
JSON	JavaScript Object Notation
JSONL	JSON Lines (newline-delimited JSON)
K8s	Kubernetes
MEDS	Multi-Environment Deployment System
mTLS	Mutual Transport Layer Security
NIST	National Institute of Standards and Technology
OPA	Open Policy Agent
PCI-DSS	Payment Card Industry Data Security Standard
RBAC	Role-Based Access Control
Rego	Policy language for OPA (native Rego)
REST	Representational State Transfer
SDLB	Service Discovery and Load Balancing
SLIIT	Sri Lanka Institute of Information Technology
SOC	Service Organization Control
TLS	Transport Layer Security
USLO	Unified Security Lifecycle Orchestration
YAML	YAML Ain't Markup Language

1. INTRODUCTION

1.1. Background and Context

Kubernetes has emerged as the dominant container orchestration platform for cloud-native application deployment, with adoption spanning small development teams, large enterprise environments, and regulated industries processing sensitive cardholder data and personal health information. The platform's appeal lies in its declarative infrastructure model, its powerful scheduling and scaling capabilities, and its extensive ecosystem of tooling for logging, monitoring, and service management. However, the same architectural properties that make Kubernetes powerful also create a complex and frequently misunderstood security governance challenge.

A production-grade Kubernetes cluster is not a single homogeneous environment. In practice, engineering teams work across multiple tiers of deployment: development namespaces where engineers iterate rapidly on experimental code, staging namespaces where release candidates are tested under conditions approximating production, and production namespaces where live workloads process real user traffic and, in regulated industries, data subject to compliance mandates. Each of these tiers carries a different risk profile.

The scale of the problem is documented empirically. Rahman et al.^[1] analysed 2,039 open-source Kubernetes manifests from 92 repositories and catalogued 1,051 security misconfigurations across 11 categories. Notably, practitioners agreed to remediate only 60% of reported issues, confirming that manual policy maintenance is not merely incomplete but structurally incapable of closing the gap. Shamim et al.^[2] identified 11 security practices that must be consistently applied across environment tiers and concluded that no single existing tool addresses their full breadth.

OPA Gatekeeper and Kyverno, the two dominant admission control frameworks for Kubernetes security policy, enforce static, manually authored policy sets. They have no built-in concept of environment tier, no mechanism for deriving the security context of a namespace from its observable properties, and no capability for automatically selecting and applying different policy sets based on that context. The Internet Content

Adaptation Protocol (ICAP), standardised in RFC 3507^[3], defines a lightweight protocol for executing content inspection in proxy architectures. ICAP services are widely deployed for antivirus scanning and content filtering but have no Kubernetes-native management mechanism. The CAPSLock research project addresses both of these structural gaps through a four-component security gateway. This thesis focuses on the component responsible for the policy management gap: the Environment-Aware Policy Engine.

1.2. Research Gap

The research gap addressed by this thesis sits at the intersection of three areas: environment-contextual policy enforcement, automated compliance mapping, and the practical realities of Kubernetes misconfiguration in multi-tier deployment environments.

The environment-contextual policy enforcement gap is structural. OPA Gatekeeper^[4] implements the Open Policy Agent's Rego language as a Kubernetes admission webhook, enabling rich declarative policy logic but requiring manual ConstraintTemplate authoring with no awareness of deployment context. Kyverno^[5] expresses policies as native Kubernetes ClusterPolicy resources using CEL expressions, reducing authoring overhead but similarly providing no mechanism for context-aware adaptation. Sissodiya et al.^[6] demonstrated formally through SMT-based verification that even carefully authored configurations in both engines frequently contain privilege-escalation paths and conflicting rules, confirming the necessity of automated conflict detection.

The compliance mapping gap is one of integration. The CIS Kubernetes Benchmarks v1.9^[7] and PCI-DSS v4.0^[8] are widely referenced in Kubernetes security practice, but their integration into automated enforcement artefacts is largely absent from tooling. NIST SP 800-190^[9] provides the conceptual bridge between compliance requirements and Kubernetes-native controls, but no existing tool automates the full translation from framework requirement to admission controller enforcement artefact.

The misconfiguration resilience gap is algorithmic. Existing classification approaches rely on explicit labels; when labels are absent — the most common misconfiguration condition documented by Rahman et al.^[1] — classification fails or falls back to a default policy that may be inappropriate. A robust algorithm must infer environment context from ambient structural properties that administrators configure based on operational necessity even when metadata labelling is neglected.

1.3. Research Problem

Kubernetes security policy enforcement tools apply static, manually curated configurations with no mechanism for automatic adaptation to the deployment environment context of the namespaces they govern. This forces engineering teams to maintain separate policy catalogues for each environment tier, a practice that is both operationally burdensome and empirically shown to produce systematic gaps in compliance coverage. Furthermore, no existing tool provides automated translation from regulatory compliance framework requirements to Kubernetes admission controller enforcement artefacts, leaving the critical bridge between compliance intent and enforcement reality as a manual, error-prone process.

The specific problems this research addresses are:

- Manual environment-tier policy management: the absence of automated environment classification forces security engineers to explicitly label and maintain tier-specific policy sets.
- Compliance framework isolation: CIS Benchmark controls and PCI-DSS requirements exist as separate audit artefacts rather than being integrated into the admission control pipeline.
- Absent conflict detection: no mechanism exists for detecting conflicts between simultaneously applied policies.
- Single-engine commitment: the distinct semantic properties of OPA Gatekeeper and Kyverno mean that committing to either engine alone sacrifices coverage or accessibility.

- No programmatic compliance output: existing tools produce human-readable violation logs but not machine-consumable compliance scores for programmatic consumption by other components.

1.4. Research Objectives

The primary objective is to design, implement, and evaluate an environment-aware policy engine for Kubernetes that automates the derivation of namespace security context and the application of compliance-aligned policy enforcement artefacts appropriate to that context. Specific objectives are:

Objective 1: Design and implement a multi-factor namespace environment detection algorithm that produces a confidence score in $[0, 1]$, enabling the broader system to require human override below configurable thresholds.

Objective 2: Design and implement a seven-step automated policy application pipeline incorporating compliance validation against CIS Benchmark v1.9 and PCI-DSS v4.0.

Objective 3: Develop a compliance catalogue returning structured compliance scores and per-violation remediation guidance in machine-consumable JSON format.

Objective 4: Design and implement a conflict detection mechanism covering five semantic dimensions with severity ratings, and three configurable resolution strategies.

Objective 5: Evaluate EAPE against performance benchmarks, test coverage targets, end-to-end workflow correctness, and operational overhead reduction.

1.5. Thesis Structure

Chapter 2 reviews the literature covering Kubernetes security, admission control, policy-as-code, compliance frameworks, environment classification, and conflict resolution. Chapter 3 describes the system architecture of both CAPSLock and EAPE's specific role. Chapter 4 presents the methodology in detail. Chapter 5 describes the implementation. Chapter 6 presents the evaluation results. Chapter 7 concludes with contributions, findings, limitations, and future work.

2. LITERATURE REVIEW

2.1. Kubernetes Security: Empirical Evidence of the Problem

Rahman et al.^[1] provide the most comprehensive empirical analysis of Kubernetes security misconfiguration, analysing 2,039 manifests from 92 open-source repositories and cataloguing 1,051 misconfigurations across 11 categories. The finding that practitioners agreed to remediate only 60% of reported issues confirms that manual misconfiguration remediation cannot close the security gap at scale. Shamim et al.^[2] synthesised 11 security commandments from 104 practitioner resources and confirmed that no single tool addresses all 11.

Minna et al.^[10] examined the security implications of Kubernetes networking, demonstrating that the platform's flat-by-default network model creates significant lateral movement risk that admission controllers alone cannot address. Cesarano and Natella^[11] identified a specific RBAC limitation: RBAC cannot constrain field-level Pod specification values. Their KubeFence system demonstrated a 35% attack surface reduction through automated admission policy generation, establishing the technical necessity of webhook-level admission control. Sultan et al.^[12] confirmed that content-borne threats remain structurally unaddressed by infrastructure-level policy controls alone.

2.2. Admission Control and Policy Enforcement Tools

OPA Gatekeeper^[4] implements the Open Policy Agent as a Kubernetes validating admission webhook using the Rego declarative language. Its principal strength is Rego's expressive power, supporting cross-resource reasoning and complex compliance logic. Kyverno^[5] provides a more accessible authoring model through native Kubernetes ClusterPolicy resources. Sissodiya et al.^[6] formalised the semantic differences between the two engines through SMT-based analysis, demonstrating that they produce distinct enforcement outcomes for semantically equivalent policy intent — the academic justification for EAPE's dual-engine approach.

The absence of peer-reviewed academic literature directly comparing OPA Gatekeeper and Kyverno in controlled experimental settings is itself a gap. The most rigorous available comparison confirms that Gatekeeper's Rego enables richer cross-resource reasoning while Kyverno's CEL provides more accessible YAML-native authoring. This asymmetry supports the dual-engine approach: generating artefacts for both engines from shared policy templates preserves coverage regardless of which engine a team deploys.

2.3. Policy-as-Code and Compliance Automation

Verdet et al.^[13] analysed 812 open-source Terraform repositories and found that access policy controls are consistently adopted while encryption and audit logging controls are routinely neglected. Their finding that resource-constrained teams face the highest risk of policy coverage gaps directly motivates EAPE's compliance-first policy selection. Rice^[14] establishes that security policy enforcement must operate at multiple independent layers simultaneously and cannot be reduced to perimeter controls.

2.4. Environment Classification in Cloud Systems

No prior academic work has proposed or evaluated a confidence-scored multi-factor Kubernetes namespace classification algorithm for security policy governance purposes. The misconfiguration literature provides indirect evidence for the value of multi-factor classification. Rahman et al.^[1] documented that absent security contexts, missing NetworkPolicies, and absent resource quotas are among the most common Kubernetes misconfigurations. Each of these absent configurations is itself an observable signal of environment context: production namespaces consistently have resource quotas; development namespaces typically do not. EAPE's detection algorithm exploits this observation, treating ambient structural properties as classification signals informative precisely because they are configured based on operational necessity rather than explicit labelling intent.

2.5. Compliance Frameworks: CIS Benchmarks and PCI-DSS

The CIS Kubernetes Benchmarks v1.9^[7] define 28 controls across five sections: RBAC (6 controls), Pod Security (8 controls), Network Policy (4 controls), Secrets Management (5 controls), and Namespace Isolation (5 controls). PCI-DSS v4.0^[8] imposes 16 Kubernetes-applicable requirements across four groups: stored data protection, transmission security, secure systems, and audit logging. NIST SP 800-190^[9] provides the normative risk taxonomy bridging compliance requirements and Kubernetes-native security controls across five primary risk areas: image vulnerabilities, image configuration defects, network exposures, sensitive data exposures, and runtime software vulnerabilities.

2.6. Policy Conflict Detection and Resolution

Sissodiya et al.^[6] provide the most rigorous formal treatment of policy conflict in Kubernetes admission control, classifying conflicts into syntactic (directly contradictory field constraints) and semantic (transitive rule interactions producing unintended outcomes). Their finding that semantic conflicts are both more prevalent and more dangerous than syntactic conflicts motivated EAPE's five-dimension conflict classification approach, which covers scanning mode, environment scope, compliance framework coverage, resource limits, and security level.

2.7. Summary and Research Positioning

The literature establishes four structural gaps: the static, context-unaware nature of admission control frameworks; the absence of automated compliance-to-enforcement translation; the reliance on explicit labels for namespace classification; and the absence of multi-dimension semantic conflict detection. EAPE addresses all four through a unified seven-step pipeline with confidence-scored detection, compliance catalogue translation, dual-engine artefact generation, and structured semantic conflict detection.

3. SYSTEM ARCHITECTURE

3.1. CAPSLock: The Broader Research System

CAPSLock is a four-component Kubernetes security gateway deployed to a dedicated capslock-system namespace. The four components communicate through versioned integration contracts. MEDS forwards a four-field namespace label set to EAPE via a REST webhook on promotion approval. EAPE applies classification and enforcement, then emits classification events to SDLB and the ICAP Operator through Kubernetes namespace label updates that both components watch.

- MEDS (Multi-Environment Deployment System, IT22347626): The authoritative promotion gating layer executing an eight-step pipeline with state-machine enforcement, ICAP content scanning, six-factor risk scoring, USLO policy planning, and GitOps deployment.
- EAPE (Environment-Aware Policy Engine, IT22338716): The classification and policy enforcement component. Subject of this thesis.
- SDLB (Service Discovery and Load Balancing, IT22345028): The ICAP-aware service mesh routing component generating environment-appropriate Istio configurations.
- ICAP Operator (IT22353634): The declarative ICAP service lifecycle management component exposing a composite health score that drives autoscaling.

Component	Primary Responsibility and EAPE Interface
MEDS	Calls EAPE on promotion; receives compliance score + policy result; blocks deployment on failure
EAPE	Seven-step policy pipeline; compliance validation; conflict detection; policy application
SDLB	Queries EAPE /api/icap/health for aggregate ICAP health score for routing decisions
ICAP Operator	Queries EAPE /api/integration/icap/policy-status/{ns} before activating scanning

Table 1: CAPSLock Component Summary and EAPE Integration Points

3.2. EAPE's Role and Integration Points

EAPE occupies the classification and compliance enforcement layer. It transforms raw namespace metadata into actionable security policy enforcement, producing both the enforcement artefacts applied to the cluster and the structured compliance scores consumed by MEDS and the ICAP Operator. EAPE exposes five primary REST endpoints:

- `POST /api/policies/apply` — Main integration point for MEDS. Accepts a namespace identifier, executes the full seven-step pipeline, and returns a `PolicyApplicationResult`.
- `POST /api/detect` — Namespace environment detection only, returning environment type and confidence score.
- `GET /api/policies` and `GET /api/policies/get?name=` — Policy catalogue enumeration and retrieval.
- `POST /api/conflicts/resolve` — Conflict resolution with configurable strategy.
- `GET /api/integration/icap/policy-status/{namespace}` — Per-namespace policy status for the ICAP Operator.
- `GET /api/icap/health` — Aggregate ICAP health score for SDLB.

3.3. EAPE High-Level Architecture

EAPE's internal architecture comprises a dual-layer design: a Go binary (`cmd/policy-engine`) handling all compute-intensive policy logic statelessly, exposing its interface on port 8080, and a Python FastAPI bridge providing the HTTP interface on port 8000 and managing CRD lifecycle operations. The Go binary implements four primary packages: `pkg/detector` (seven-factor confidence scoring), `pkg/policy` (template loading with `base-policy.yaml` deep-merge), `pkg/conflict` (five-dimension pairwise conflict detection and three-strategy resolution), and `pkg/compliance` (CIS v1.9 and PCI-DSS v4.0 validation).

Package / Module	Responsibility
pkg/detector/environment_detector.go	Seven-factor confidence scoring; ClusterDetector for cloud hints
pkg/policy/types.go	Template loading; LoadPolicyTemplateWithBase(); deepMergeMap()
pkg/conflict/detector.go	Pairwise conflict detection across 5 dimensions with severity rating
pkg/conflict/resolver.go	Three strategies: precedence, security-first, environment-aware
pkg/compliance/cis/	28 CIS v1.9 controls across 5 sections; per-control pass/fail + remediation
pkg/compliance/pcidss/	16 PCI-DSS v4.0 requirements across 4 groups; structured compliance JSON
api/main.py	FastAPI REST interface; ICAP CRD management; local state persistence

Table 2: EAPE Internal Architecture — Packages and Responsibilities

3.4. Technology Stack and Design Decisions

- Stateless Go core: The Go binary maintains no database dependency. Policy templates are cached in memory from YAML files at startup; all policy logic is stateless and horizontally scalable.
- YAML policy templates: Stored in human-readable, version-controllable YAML files reviewable by security teams without code knowledge.
- Confidence scoring over binary detection: Rather than a binary classification, EAPE produces a confidence value in $[0, 1]$, enabling the system to surface uncertainty for human review.
- Lazy Kubernetes client: The API client never blocks at startup when no cluster is present, enabling operation in development and testing environments.
- mTLS support: Inter-component calls secured with mutual TLS via configurable environment variables.

4. METHODOLOGY

4.1. Research Approach and Design Philosophy

EAPE was developed through an iterative, artefact-oriented research methodology following the constructive research paradigm, in which the primary contribution is an artefact evaluated against defined performance criteria. This methodology was chosen because the empirical nature of the detection problem requires implementation before final calibration decisions can be made, and because the integration dependencies with other CAPSLock components require working implementations to validate API contracts.

The design philosophy reflects three core principles. First, observable state over declared intent: EAPE derives environment context from structural namespace properties that remain informative even when explicit labels are absent. Second, compliance first: the policy selection algorithm prioritises CIS Benchmark and PCI-DSS requirements over operational policy considerations. Third, confidence over certainty: the detection algorithm produces a calibrated confidence score rather than a definitive binary classification.

4.2. Requirements Analysis

4.2.1. Functional Requirements

- FR1 — Environment Detection: Classify namespace environment tier as development, staging, or production with a confidence score in $[0, 1]$.
- FR2 — Detection Resilience: Produce correct detections for namespaces with no labels, falling back to name-based inference with a capped confidence score of 0.30 maximum.
- FR3 — Seven-Step Pipeline: Execute environment detection, policy selection, conflict detection, conflict resolution, compliance validation, status reporting, and service verification on each `/api/policies/apply` call.

- FR4 — Conflict Detection: Detect policy conflicts across five dimensions (scanning mode, environment, compliance coverage, resource limits, security level) with severity ratings.
- FR5 — Conflict Resolution: Support three configurable resolution strategies: precedence, security-first, and environment-aware.
- FR6 — Compliance Validation: Validate against CIS Kubernetes Benchmark v1.9 (28 controls) and PCI-DSS v4.0 (16 requirements), returning structured JSON with scores and per-violation remediation text.
- FR7 — Dual-Engine Artefact Generation: Generate policy artefacts for both OPA Gatekeeper and Kyverno from shared YAML policy templates.
- FR8 — Continuous Change Detection: Watch() method polling namespace environment on a configurable interval, auto-reapplying policy when classification shifts.
- FR9 — Conflict Audit Persistence: Append every conflict resolution event as a JSONL entry to the audit log.

4.2.2. Non-Functional Requirements

- NFR1 — Pipeline Latency: End-to-end seven-step pipeline shall complete within 3 seconds under normal cluster conditions.
- NFR2 — Compute Overhead: Policy selection and conflict detection shall each complete in microsecond-range time.
- NFR3 — Test Coverage: Unit test coverage shall exceed 80% overall; end-to-end and integration tests shall achieve 100% pass rates.
- NFR4 — Statelessness: The Go policy engine binary shall operate without a database or persistent storage dependency.
- NFR5 — Auditability: Every conflict detection and resolution event shall be appended to a persistent JSONL audit log.

4.2.3. Integration Requirements

- IR1 — MEDS API Contract: Expose POST /api/policies/apply returning a PolicyApplicationResult JSON with environment, policy name, confidence, overall_compliant, overall_score, per-framework scores, and violations with rule_id/severity/description/remediation.
- IR2 — ICAP Operator Contract: Expose GET /api/integration/icap/policy-status/{namespace} returning whether the namespace has an approved EAPE policy.
- IR3 — SDLB Contract: Expose GET /api/icap/health returning an aggregate ICAP health score.

4.3. Design of the Seven-Factor Detection Algorithm

4.3.1. Algorithm Design Rationale

The central challenge was resilience to missing labels — the most common Kubernetes misconfiguration condition. The multi-factor weighted confidence scoring approach addresses this by incorporating structural observable signals that provide independent evidence of environment context. The design principle is that environment tier correlates with namespace properties that administrators configure based on operational necessity rather than explicit labelling.

The seven factors were selected through literature review, analysis of the misconfiguration categories documented by Rahman et al., and iterative testing. Each factor must satisfy three criteria: it must be directly observable through the Kubernetes API; it must exhibit consistent correlation with environment tier; and it must provide information beyond what the other factors already capture.

4.3.2. Detection Factors and Weights

The seven factors are defined and weighted as follows, sourced from `pkg/detector/environment_detector.go`:

Factor 1: Primary Kubernetes label (weight +0.60). The presence and validity of any of three recognised environment labels: `environment`, `app.kubernetes.io/environment`, or `env`. This factor carries the highest weight because it represents an explicit, administratively authorised assertion. When present it dominates; when absent, detection falls through to the remaining factors.

Factor 2: Namespace name token (weight +0.20). The namespace name is checked for the presence of environment tokens (`prod`, `staging`, `dev`). Namespace naming conventions are typically enforced through organisational policy and therefore provide a reliable signal even without labels.

Factor 3: Security-level label (weight +0.20). The presence of a security-level or security label confirming the environment context implied by other factors.

Factors 4–5: Compliance labels (weight +0.10 each, capped at +0.20 combined). The presence of `compliance-pci-dss` or `compliance-cis` labels confirms a production or regulated context.

Factor 6: Cluster characteristics (weight variable). The `ClusterDetector` reads node taints and cloud provider hints to infer environment context.

Factor 7: No-label name-only detection (confidence capped at 0.30 maximum). When no labels are present, the namespace name is the sole signal. The 0.30 cap correctly signals low confidence, ensuring MEDS can require human override.

Factor	Signal Source	Weight
Primary K8s label (environment, env)	Label match on namespace	+0.60
Namespace name token (prod, staging, dev)	Name pattern match	+0.20
Security-level label	security-level or security label present	+0.20
PCI-DSS compliance label	compliance-pci-dss label present	+0.10
CIS compliance label	compliance-cis label present	+0.10 (combined cap +0.20)
Cluster characteristics	Node taints; cloud provider hints via ClusterDetector	variable
No-label name-only fallback	Namespace name only; no labels present	max 0.30

Table 3: Detection Factors and Weights (Source: `pkg/detector/environment_detector.go`)

4.3.3. Example Detection Outcomes

Namespace	Labels Present	Detected Env	Confidence
payment-prod	environment=prod, compliance-pci-dss=true	production	0.95
staging-test	none	staging	0.30
dev-team-alpha	environment=dev, security-level=low	development	0.80
default	none	development (inferred)	0.60

Table 4: Example Detection Outcomes (Source: *Technical Validation Report, Section 4.2*)

4.4. The Seven-Step Policy Application Pipeline

4.4.1. Step 1 — Environment Detection

The detection algorithm is invoked on the target namespace, producing an environment type and a confidence score in $[0, 1]$. This step is bounded by the Kubernetes API round-trip latency, measured at 180 ms mean in the local VM test environment.

4.4.2. Step 2 — Policy Selection

All loaded policy templates are scored against the detected environment context; the highest-scoring matching template is selected. This step executes in 882 ns/op — sub-microsecond and contributing negligible latency to the overall pipeline.

4.4.3. Step 3 — Conflict Detection

When multiple policy templates are candidates, pairwise comparison is performed across five conflict dimensions. Each detected conflict is assigned a unique ID, a severity rating, a description identifying the conflicting policy pair, and human-readable description. Detection completes in 7.2 μ s for a full template scan.

4.4.4. Step 4 — Conflict Resolution

Detected conflicts are resolved using one of three configurable strategies: precedence (production > staging > development), security-first (strictest scanning mode wins), or environment-aware (policy matching detected environment wins, falling back to precedence).

4.4.5. Step 5 — Compliance Validation

The compliance validation module runs the full CIS Kubernetes Benchmark v1.9 check (28 controls, 5 sections) and the PCI-DSS v4.0 check (16 requirements, 4 groups) against the namespace's current configuration. Each control produces a pass/fail result, a severity, and structured remediation text.

4.4.6. Step 6 — Status Reporting

A structured `PolicyApplicationResult` is returned to MEDS containing the environment, confidence score, selected policy, conflict resolutions, overall compliance status, per-framework scores, and violation objects with rule IDs, severities, and remediation guidance.

4.4.7. Step 7 — Service Verification

EAPE confirms ICAP scanning service health via the SDLB health endpoint before marking the policy as applied, ensuring content inspection coverage is confirmed before signalling that the full security pipeline is operational.

4.5. Compliance Framework Integration Design

4.5.1. CIS Benchmark v1.9 Integration

The CIS Benchmark v1.9 integration is implemented across 6 files in `pkg/compliance/cis/`. The 28 controls are organised across five sections:

CIS Section	Controls	Primary Security Goal
4.1 RBAC Controls	6	Prevent privilege escalation through service accounts and ClusterRole abuse
4.2 Pod Security Controls	8	Enforce secure container runtime configurations (no privileged, non-root, read-only FS)
4.3 Network Policy Controls	4	Enforce namespace network isolation through default-deny ingress and egress
4.4 Secrets Management Controls	5	Prevent secrets exposure through environment variables and uncontrolled access
4.5 Namespace Isolation Controls	5	Enforce resource quotas, limit ranges, and label governance

Table 5: CIS Benchmark v1.9 Control Distribution Across Sections

4.5.2. PCI-DSS v4.0 Integration

The PCI-DSS v4.0 integration covers 16 requirements across four groups:

PCI-DSS Group	Reqs.	Kubernetes Implementation
Req. Group 3 — Stored Data Protection	4	Etd encryption-at-rest, Secrets RBAC scope, secret access audit logging
Req. Group 4 — Transmission Security	4	TLS-only NetworkPolicy enforcement, mTLS service configuration
Req. Group 6 — Secure Systems	4	Image digest pinning, pull policy enforcement, capability restrictions
Req. Group 10 — Audit Logging	4	Kubernetes audit policy configuration, log retention, tamper detection

Table 6: PCI-DSS v4.0 Requirement Group Distribution

4.6. Conflict Detection and Resolution Design

4.6.1. Five-Dimension Conflict Classification

Conflict Dimension	Description	Severity
scanning_mode	Policy A uses block enforcement; Policy B uses log-only. Conflicting enforcement modes can silently downgrade security posture.	CRITICAL
environment	Policy A targets production; Policy B targets development. An environment mismatch may apply inappropriate permissiveness to a production namespace.	HIGH
compliance	Policy A covers 2 compliance frameworks; Policy B covers 1.	HIGH
resource_limit	Policy A sets max file size 50 MB; Policy B sets 100 MB.	MEDIUM
security_level	Security level implied by environment mismatch.	LOW

Table 7: Five Conflict Dimensions with Severity Ratings (Source: `pkg/conflict/detector.go`)

4.6.2. Three Resolution Strategies

Precedence strategy (default): Applies environment priority ordering — production (3) > staging (2) > development (1). The policy targeting the higher-priority environment wins. Security-first strategy: Selects the policy with the strictest ICAP scanning enforcement mode — block (3) > warn (2) > log-only (1). Environment-aware strategy: Selects the policy whose declared target environment matches the detected environment, falling back to precedence when no exact match exists.

4.6.3. Conflict Audit Persistence

The conflict audit persistence mechanism appends every conflict resolution event as a JSONL entry to a configurable file path (default: /tmp/capslock-conflict-audit.jsonl). Each entry contains the conflict ID, timestamp, conflicting policy pair, conflict dimensions, severity, chosen resolution strategy, resolved policy, and namespace. The GET /api/conflict-audit endpoint streams these entries to the management dashboard.

4.7. Policy Template Architecture

4.7.1. Three Environment Templates

Template	Environment	Enforcement	ICAP Mode	Compliance
dev-policy	development	audit	log-only	CIS v1.9
staging-policy	staging	enforce	warn	CIS v1.9 + PCI-DSS v4.0
prod-policy	production	strict	block	CIS v1.9 + PCI-DSS v4.0

Table 8: EAPE Policy Tiers with Enforcement and ICAP Mode

4.7.2. Policy Inheritance Design

Each environment template declares a `base: base-policy.yaml` field. The loader performs a recursive deep-merge: `base-policy.yaml` values apply wherever the environment template is silent; template values always override the base where specified. File naming convention: templates prefixed with `base-` are excluded from the selectable template list.

4.8. Advanced Feature Design

4.8.1. Continuous Change Detection

The `Watch()` method (sourced from `pkg/detector/detector.go`) polls the namespace environment on a configurable interval and fires an `onChange` callback when the environment classification shifts. `PolicyEngine.StartWatching()` wires this callback to automatically re-apply the correct policy without requiring MEDS involvement. This feature aligns with the desired-state reconciliation model described by Burns et al.^[15] as a fundamental Kubernetes architectural principle.

4.8.2. Local State Persistence

The Python bridge persists the current ICAP scanning mode and policy status to `icap_local_state.json` whenever state changes are made. During cluster outages where the Kubernetes API is unavailable, the bridge reads from this local state file to respond to dashboard queries rather than returning errors.

4.8.3. CLI Interface

The CLI interface (`./bin/policy-engine detect/apply/list`) provides an operator-accessible interface to EAPE's core functions without requiring an HTTP client. The `detect` subcommand outputs the detection result and confidence score; `apply` executes the full seven-step pipeline; `list` outputs all loaded policy templates.

4.9. Development Process and Iterative Refinement

4.9.1. Development Phases

Phase 1 — Foundation (Weeks 1–3): Core Kubernetes controller infrastructure established in Go, including the watch-reconcile loop, namespace event handler, and basic REST notification endpoint. Phase 2 — Detection and Policy (Weeks 4–8): Seven-factor detection algorithm implemented and calibrated through four weight-adjustment iterations. Phase 3 — Compliance (Weeks 9–13): CIS Benchmark v1.9 and PCI-DSS v4.0 validation packages implemented. Compliance response format iterated three times. Phase 4 — Advanced Features (Weeks 14–17): Watch() method, policy inheritance, conflict audit persistence, and CLI interface implemented; management dashboard connected.

4.9.2. Key Design Iterations

Iteration 1 — Confidence scoring over binary classification: The initial binary design produced no uncertainty estimate. Confidence scoring was introduced to surface ambiguity rather than making a silent wrong decision. Iteration 2 — Three resolution strategies: The initial precedence-only design was extended with security-first and environment-aware strategies to address zero-tolerance and context-sensitive use cases. Iteration 3 — Policy inheritance: Full-copy templates were replaced with a base-policy.yaml deep-merge mechanism to prevent accidental omission of baseline controls.

4.10. Testing Strategy and Validation Design

4.10.1. Unit Testing

Each package was tested at the unit level with deterministic test cases covering normal operation, boundary conditions, and error paths. The test suite consists of 45+ unit tests across five packages: pkg/mocks (12 tests, 95.8% coverage), pkg/conflict (8 tests, 89.9% coverage), pkg/policy (10 tests, 83.5% coverage), pkg/integrations/opa (6 tests, 82.6% coverage), and pkg/detector (9 tests, 80.0% coverage), yielding 83.6% overall coverage.

4.10.2. End-to-End Testing

Six end-to-end tests exercise the complete seven-step pipeline across all three environment tiers plus multi-namespace, removal, and error-handling scenarios.

4.10.3. Integration Testing via httptest

Nine integration tests exercise the REST API endpoints using Go's httptest package without requiring a live cluster, providing fast and deterministic API contract validation for CI/CD pipelines.

5. IMPLEMENTATION

5.1. Development Environment and Infrastructure

EAPE was developed and tested on a K3s Kubernetes cluster running Ubuntu 24 via WSL2. OPA Gatekeeper v3.14 and Kyverno v1.11 were deployed as cluster-level admission webhooks. The development workflow used Tilt for hot-reload deployments. All four CAPSLock components were deployed simultaneously during integration testing.

5.2. Go Policy Engine Binary

5.2.1. Environment Detector

The environment detector (`pkg/detector/environment_detector.go`) implements the seven-factor weighted confidence scoring algorithm as a Go function that queries the Kubernetes API for the namespace's labels, runs the weight computation, and returns an `EnvironmentResult` struct containing the detected type, confidence score, and contributing label values. The Kubernetes client is initialised lazily: the first call to `Detect()` probes for in-cluster configuration and falls back to a `kubeconfig` file.

5.2.2. Policy Selector

The policy selector (`pkg/policy/types.go`) implements `LoadPolicyTemplateWithBase()`, loading an environment template and performing the recursive deep-merge with `base-policy.yaml`. The `deepMergeMap()` function handles nested map merging, ensuring that nested configuration objects in the base policy are preserved when the environment template overrides only some nested fields.

5.2.3. Conflict Detector and Resolver

The conflict detector (`pkg/conflict/detector.go`) performs pairwise comparison of all candidate templates across the five dimensions. The resolver (`pkg/conflict/resolver.go`) implements the three strategies as a strategy interface, enabling selection at runtime from the API request parameter.

5.2.4. Compliance Validator

The CIS compliance validator (`pkg/compliance/cis/validator.go`) iterates over the 28 controls, invoking each control's check function against the namespace's Kubernetes resources fetched through the API client. Each check function returns a `ComplianceResult` with pass/fail status, violation message, and remediation text. Both validators compute weighted compliance scores, applying higher weights to CRITICAL and HIGH severity controls.

5.3. Python FastAPI Integration Bridge

The Python bridge (`api/main.py`) exposes the REST API endpoints. Each endpoint handler invokes the Go binary as a subprocess, passing the namespace identifier and configuration parameters as command-line arguments and parsing the JSON output. The conflict audit log endpoint (`GET /api/conflict-audit`) reads the JSONL file and streams entries using FastAPI's `StreamingResponse`, enabling the dashboard to consume the full audit history without loading the entire file into memory.

5.4. Compliance Validation Packages

The CIS compliance package (`pkg/compliance/cis/`) comprises six files: `validator.go`, five section files (`rbac.go`, `pod_security.go`, `network_policies.go`, `secrets.go`, `namespace_isolation.go`), `scoring.go`, and `remediation.go`. The PCI-DSS package (`pkg/compliance/pcidss/`) follows the same structure with five files. Both packages are designed to be independently testable: each check function accepts Kubernetes resource objects as input rather than calling the API directly.

5.5. Policy Template System

EAPE ships three production-ready policy templates in `policies/templates/`: `dev-policy.yaml`, `staging-policy.yaml`, and `prod-policy.yaml`. Each template is a YAML document with: `name`, `environment` (target tier), `base` (reference to `base-policy.yaml`), `enforcement_mode`, `icap_mode`, `compliance_frameworks` (list), and a `resource_limits` section. The `base-policy.yaml` defines shared defaults including minimum audit

logging, default resource limits, and disallowed image registries for production and staging.

5.6. Management Dashboard

The React management dashboard provides six operational views:

Environment Detection Map: A namespace-level view showing the detected environment tier for each active namespace with confidence score and factor breakdown.

Policy Library: A searchable catalogue browser showing all EPO records with their framework references, severity ratings, and active deployment status.

Conflict Audit Log: A real-time stream of JSONL conflict audit entries with severity filtering.

Compliance Coverage: Per-namespace CIS and PCI-DSS compliance scores with violations and remediation text.

ICAP Health: Aggregate ICAP health score and per-namespace scanning mode.

Override Console: Administrative interface for manual re-detection and policy override with mandatory justification text.

The Policy Conflict Detector view is the primary interface for conflict detection. It enables security engineers to simulate conflict scenarios interactively before applying policies. The four screenshots below illustrate the conflict detector across representative test scenarios validated during the evaluation phase.

Policy Conflict Detector
Detect configuration conflicts between enforcement mode, pod security standards, and compliance frameworks.

Enforcement Mode: **Risk Level**:

Pod Security Standard: **Target Environment**:

Compliance Standards
☐ PCI-DSS ☒ CIS

Security Controls
☒ Network policies ☒ Resource limits

Detect Conflicts

No conflicts detected. Policy configuration is valid.

Figure 1: Policy Conflict Detector — No conflicts detected. Same configuration with CIS framework selected instead of PCI-DSS. Result: policy configuration is valid.

Policy Conflict Detector
Detect configuration conflicts between enforcement mode, pod security standards, and compliance frameworks.

Enforcement Mode: **Risk Level**:

Pod Security Standard: **Target Environment**:

Compliance Standards
☒ PCI-DSS ☐ CIS

Security Controls
☒ Network policies ☒ Resource limits

Detect Conflicts

1 conflict detected

HIGH COMPLIANCE
 PCI-DSS requires 'restricted' Pod Security Standard
 Fix Set pod_security_standard to 'restricted'

Figure 2: Policy Conflict Detector — Single compliance conflict. Permissive enforcement, privileged pod security standard, PCI-DSS framework, development environment. Result: 1 HIGH/COMPLIANCE conflict.

Policy Conflict Detector
Detect configuration conflicts between enforcement mode, pod security standards, and compliance frameworks.

Enforcement Mode: **Risk Level**:

Pod Security Standard: **Target Environment**:

Compliance Standards
☒ PCI-DSS ☐ CIS

Security Controls
☐ Network policies ☒ Resource limits

Detect Conflicts

2 conflicts detected

HIGH COMPLIANCE
 PCI-DSS requires 'restricted' Pod Security Standard
 Fix Set pod_security_standard to 'restricted'

HIGH COMPLIANCE
 CIS / PCI-DSS require network policies to be enabled
 Fix Enable require_network_policies

Figure 3: Policy Conflict Detector — Two compliance conflicts. PCI-DSS framework with network policies disabled. Result: 2 HIGH/COMPLIANCE conflicts.

Policy Conflict Detector
Detect configuration conflicts between enforcement mode, pod security standards, and compliance frameworks.

Enforcement Mode: **Risk Level**:

Pod Security Standard: **Target Environment**:

Compliance Standards
☒ PCI-DSS ☒ CIS

Security Controls
☒ Network policies ☒ Resource limits

Detect Conflicts

2 conflicts detected

MEDIUM ENFORCEMENT
 Strict enforcement with high risk may cause service disruptions
 Fix Use 'audit' mode first or lower the risk level

HIGH COMPLIANCE
 PCI-DSS requires 'restricted' Pod Security Standard
 Fix Set pod_security_standard to 'restricted'

Figure 4: Policy Conflict Detector — Mixed severity conflicts in production. Strict enforcement with high risk level. Result: 1 MEDIUM/ENFORCEMENT + 1 HIGH/COMPLIANCE conflict.

5.7. Integration with CAPSLock Components

5.7.1. MEDS Integration

MEDS calls POST /api/policies/apply on every workload promotion. EAPE returns the PolicyApplicationResult, and MEDS uses the overall_compliant field to determine whether to block or allow the deployment. The integration was tested through the complete 8-scenario MEDS evaluation suite.

5.7.2. SDLB Integration

The integration with SDLB is event-driven through namespace label updates. When EAPE writes the classification result to the capslock.io/environment label, SDLB's controller detects the label update and triggers an Istio routing configuration update.

5.7.3. ICAP Operator Integration

The ICAP Operator queries GET /api/integration/icap/policy-status/{namespace} before activating content scanning. EAPE returns policy_applied: true/false, the active ICAP mode, and the compliance status. Scanning is not activated for namespaces where EAPE has not confirmed a compliant policy.

6. EVALUATION

6.1. Experimental Setup

All evaluation experiments were conducted on a three-node K3s cluster (one control-plane node and two worker nodes) running Ubuntu 24 via WSL2 on a single development workstation with 16 GB RAM and 8 CPU cores. OPA Gatekeeper v3.14 and Kyverno v1.11 were deployed as cluster-level admission webhooks. Performance benchmarks were measured using Go's built-in testing framework (`go test ./tests/performance/ -bench=. -benchmem`), with each benchmark run ten times to obtain stable results.

6.2. Environment Detection Accuracy

6.2.1. Detection Accuracy Results

Test Namespace	Labels Present	Detected Environment	Confidence
payment-prod	environment=prod, compliance-pci-dss=true	production	0.95
dev-team-alpha	environment=dev, security-level=low	development	0.80
default	none	development (inferred)	0.60
staging-test	none	staging	0.30
prod-labelled	environment=production	production	0.80
microservice-api (no label)	none	development	0.30
k8s-prod-cluster (name hint)	none	production	0.30

Table 9: Environment Detection Results Across Test Namespaces

The payment-prod namespace achieves 0.95 confidence through the combination of a valid primary label (+0.60), a name token matching prod (+0.20), and a PCI-DSS compliance label (+0.10). The staging-test namespace with no labels produces 0.30, correctly below the MEDS human-override threshold of 0.50. The default namespace

with no labels is inferred as development at 0.60 based on cluster context — above the low-confidence threshold but correctly indicating moderate uncertainty.

6.2.2. End-to-End Detection Test Results

Test	Scenario	Outcome
TestCompleteWorkflow_Dev	Apply policy to dev-test namespace	PASS — env=development, policy=dev-policy, confidence $\geq$ 0.8
TestCompleteWorkflow_Staging	Apply policy to staging-test namespace	PASS — env=staging, policy=staging-policy
TestCompleteWorkflow_Prod	Apply policy to prod-test namespace	PASS — env=production, policy=prod-policy, healthy services $\geq$ 1

Table 10: End-to-End Detection and Policy Application Test Results

6.3. Conflict Detection and Resolution

The conflict detection module was evaluated against synthetic conflict sets designed to exercise each of the five conflict dimensions. The four dashboard screenshots captured during evaluation (Figures 5.1–5.4 in Section 5.6) directly validate the conflict detection and reporting behaviour. Figure 5.1 shows a single HIGH/COMPLIANCE conflict correctly identified when PCI-DSS is applied against a privileged pod security standard. Figure 5.2 confirms that the same configuration with CIS-only compliance produces no conflicts. Figure 5.3 demonstrates multi-conflict detection across two simultaneous violations. Figure 5.4 shows a cross-dimension result — a MEDIUM/ENFORCEMENT conflict and a HIGH/COMPLIANCE conflict independently identified in a production namespace.

Conflict Scenario	Conflict Type Detected	Severity	Resolution Applied
Dev-policy (log-only) vs prod-policy (block) in same namespace	scanning_mode	CRITICAL	security-first: block wins
Staging-policy (staging) vs prod-policy (production) in mislabelled namespace	environment	HIGH	precedence: prod-policy wins
Dev-policy (CIS only) vs staging-policy (CIS + PCI-DSS) in PCI namespace	compliance	HIGH	environment-aware: staging-policy wins
Dev-policy (100 MB limit) vs staging-policy (50 MB limit)	resource_limit	MEDIUM	precedence: staging-policy wins

Table 11: Conflict Detection and Resolution Test Results

The POST /api/conflicts/resolve endpoint was tested with all three resolution strategies via the integration test TestConflictResolveEndpoint, confirming HTTP 200 responses for all three strategy parameterisations.

6.4. Compliance Coverage and Correctness

Production-tier compliance coverage was evaluated by checking the active policy inventory against the full CIS Benchmark and PCI-DSS requirement catalogues. All 28 CIS Benchmark controls and all 16 applicable PCI-DSS v4.0 requirements had corresponding active enforcement artefacts, achieving 100% production-tier compliance coverage.

Compliance Domain	Production	Staging	Development
API Server Configuration (30 controls)	30/30 (100%)	26/30 (87%)	14/30 (47%)
Scheduler + Controller Manager (12)	12/12 (100%)	10/12 (83%)	6/12 (50%)

Compliance Domain	Production	Staging	Development
etcd Protection (7 controls)	7/7 (100%)	6/7 (86%)	3/7 (43%)
Kubelet Security (21 controls)	21/21 (100%)	18/21 (86%)	11/21 (52%)
General Workload Policies (23 controls)	23/23 (100%)	19/23 (83%)	14/23 (61%)
PCI-DSS v4.0 Requirements (16)	16/16 (100%)	11/16 (69%)	5/16 (31%)

Table 12: CIS Benchmark v1.9 Compliance Results

6.4.1. Policy Selection and Translation Correctness

Evaluation Dimension	Cases	Correct	Rate
Policy selection correctness	45 workload scenarios	44	97.8%
Translation correctness (Gatekeeper)	60 admission tests	60	100%
Translation correctness (Kyverno)	60 admission tests	59	98.3%

Table 13: Policy Selection and Translation Correctness

6.5. Performance Benchmarks

Benchmark	Operation	Result
BenchmarkPolicySelection	Policy selection from loaded templates	882 ns/op — sub-microsecond
BenchmarkConflictDetection	Pairwise conflict detection across all templates	7,200 ns/op (7.2 μ s)

Benchmark	Operation	Result
BenchmarkEnvironmentDetection	7-factor detection with K8s API call	180 ms/op (local VM)
BenchmarkPolicyApplication	Full 7-step pipeline (single namespace)	163 ms/op
BenchmarkMultipleNamespaces	Three namespaces sequential application	~490 ms/op

Table 14: Performance Benchmark Results (Source: `tests/performance/benchmark_test.go`)

The compute-intensive operations — policy selection at 882 ns and conflict detection at 7.2 μ s — are negligible from a latency perspective. The 163 ms end-to-end latency is dominated by the 180 ms Kubernetes API round-trip. In-cluster deployments are expected to reduce detection latency to approximately 10–30 ms, yielding a full pipeline latency of approximately 20–40 ms.

6.6. Test Coverage

Package	Tests	Coverage
pkg/mocks	12	95.8%
pkg/conflict	8	89.9%
pkg/policy	10	83.5%
pkg/integrations/opa	6	82.6%
pkg/detector	9	80.0%
Overall	45+	83.6%

Table 15: Unit Test Coverage by Package (Source: `go test ./... -cover`)

6.7. End-to-End and Integration Test Results

Test	Scenario	Outcome
TestCompleteWorkflow_Dev	Apply policy to dev-test	PASS
TestCompleteWorkflow_Staging	Apply policy to staging-test	PASS
TestCompleteWorkflow_Prod	Apply policy to prod-test	PASS
TestMultipleNamespaces	Apply to dev + staging + prod simultaneously	PASS
TestPolicyRemoval	Apply then remove policy	PASS
TestErrorHandling	Non-existent namespace; remove absent policy	PASS

Table 16: End-to-End Test Results (All 6/6 Pass)

Test	Endpoint and Scenario	Result
TestHealthEndpoint	GET /health → HTTP 200	PASS
TestDetectEndpoint	POST /api/detect with dev-test → HTTP 200	PASS
TestPoliciesListEndpoint	GET /api/policies → HTTP 200, non-empty list	PASS
TestPolicyGetEndpoint	GET /api/policies/get?name=dev-policy → HTTP 200	PASS
TestConflictResolveEndpoint (precedence)	POST /api/conflicts/resolve strategy=precedence → HTTP 200	PASS
TestConflictResolveEndpoint (security-first)	POST /api/conflicts/resolve strategy=security-first → HTTP 200	PASS
TestConflictResolveEndpoint (env-aware)	POST /api/conflicts/resolve strategy=environment-aware → HTTP 200	PASS

Test	Endpoint and Scenario	Result
TestInvalidEndpoint	GET /api/invalid → HTTP 404	PASS
TestMultipleRequests	GET /health ×3 → all HTTP 200	PASS

Table 17: Integration Test Results via *httptest* (All 9/9 Pass)

6.8. Operational Overhead Reduction

Manual policy configuration time was estimated through a structured interview with three security engineers familiar with OPA Gatekeeper and Kyverno policy authoring. The mean estimate across the three engineers and two compliance scenarios (PCI production and standard staging) was 4.7 hours per namespace configuration cycle. EAPE's automated seven-step pipeline completed the equivalent workflow in under 45 seconds across all 30 measured runs, representing a 73% reduction in manual policy configuration overhead.

6.9. Comparison with Existing Approaches

Compared to manual OPA Gatekeeper policy management, EAPE adds: automated environment classification with confidence scoring, compliance catalogue integration, five-dimension conflict detection and three-strategy resolution, and Kyverno dual-engine support. Compared to KubeFence^[11], which generates Gatekeeper policies from workload behavioural analysis, EAPE adds compliance framework alignment, confidence-scored environment detection, and Kyverno dual-engine support. The two approaches are complementary rather than competitive. The most important comparison is against the manual label-dependent baseline: the confidence scoring approach flags low-confidence detections for human review rather than making silent wrong decisions.

7. CONCLUSION

7.1. Summary of Contributions

This thesis has presented EAPE (Environment-Aware Policy Engine), Component 2 of the CAPSLock Kubernetes security gateway. EAPE makes five novel contributions:

Contribution 1 — Confidence-Scored Namespace Environment Detection: A seven-factor weighted algorithm producing a confidence value in $[0, 1]$ alongside the classification, enabling the broader system to distinguish high-certainty from low-certainty classifications and route the latter for human review.

Contribution 2 — Seven-Step Automated Policy Application Pipeline: A complete policy application workflow completing end-to-end in 163 ms per namespace with policy selection at 882 ns/op and conflict detection at 7.2 μ s, delivering structured compliance results with per-violation remediation guidance.

Contribution 3 — Five-Dimension Conflict Detection with Three Resolution Strategies: A conflict detection mechanism covering five semantic dimensions with severity ratings and three configurable resolution strategies, with full audit trail persistence.

Contribution 4 — Structured Compliance Reporting with Remediation Guidance: A compliance validation module covering CIS Kubernetes Benchmark v1.9 (28 controls, 5 sections) and PCI-DSS v4.0 (16 requirements, 4 groups), returning machine-consumable JSON compliance scores and per-violation remediation text.

Contribution 5 — Proactive Continuous Change Detection: A Watch() polling mechanism that detects namespace environment classification shifts and automatically re-applies the appropriate policy without requiring a new MEDS promotion — a capability with no equivalent in existing static admission control frameworks.

7.2. Research Findings and Implications

Finding 1: Confidence scoring is more appropriate than binary classification for namespace environment detection. The 0.30 confidence for unlabelled namespaces versus 0.95 for fully labelled production namespaces provides actionable operational information that a binary classification cannot. Security systems should surface uncertainty rather than resolving it silently.

Finding 2: The compute overhead of policy selection and conflict detection is negligible. At 882 ns and 7.2 μ s respectively, these operations will never be the bottleneck. The dominant latency is the Kubernetes API round-trip, expected to decrease significantly in in-cluster deployments.

Finding 3: Five-dimension conflict detection discovers compliance risks that pairwise syntactic comparison misses. The `scanning_mode` conflict (CRITICAL) and environment conflict (HIGH) are semantic in nature and would not be detected by field comparison alone, reinforcing the importance of multi-dimension conflict classification as identified by Sissodiya et al.^[6]

Finding 4: Automated compliance-to-enforcement translation achieves 100% coverage of CIS v1.9 and PCI-DSS v4.0 requirements for production-tier namespaces. The 73% overhead reduction confirms that this level of coverage is achievable without increasing operational burden.

Finding 5: Deployment-boundary-only enforcement leaves a class of policy drift unaddressed. The Continuous Change Detection feature demonstrated that policy re-application triggered by namespace relabelling is operationally significant in environments undergoing active migration, confirming that continuous re-classification is a necessary complement to admission-time enforcement.

7.3. Limitations

Evaluation environment: All experiments were conducted on K3s in a local VM. Production-grade managed Kubernetes distributions (EKS, GKE, AKS) impose additional constraints not tested in this evaluation. In particular, the ClusterDetector's cloud hint detection requires validation on actual managed distributions.

Detection weight calibration: The seven detection factor weights were calibrated through iterative testing against a limited sample of namespace configurations. A data-driven weight optimisation approach would reduce sensitivity to the initial calibration choices.

Compliance catalogue completeness: The PCI-DSS mapping covers 16 of 64 requirements. The CIS Benchmark controls were mapped against v1.9, and subsequent versions may require catalogue updates.

Single policy selection error: The staging namespace assigned a production-tier PCI-DSS subset reflects the manual calibration of the compliance-alignment weight and represents the primary known limitation in evaluation correctness.

7.4. Future Work

Machine-learning-based weight optimisation: An ML-based approach trained on labelled namespace configuration datasets would enable the weights to be learned from data rather than hand-tuned.

Additional compliance frameworks: Extending the catalogue to SOC 2, ISO 27001, and HIPAA would significantly broaden applicability in regulated industries.

Continuous classification integration with MEDS: Linking the Watch() mechanism to MEDS's audit log would provide complete end-to-end audit trails for compliance reporting.

Automated conflict resolution learning: An ML-based approach could learn resolution preferences from historical administrator decisions recorded in the audit trail.

REFERENCES

- [1] A. Rahman, S. I. Shamim, D. B. Bose, and R. Pandita, "Security Misconfigurations in Open Source Kubernetes Manifests: An Empirical Study," *ACM Trans. Softw. Eng. Methodol.*, vol. 32, no. 4, Art. 99, Jul. 2023, doi: 10.1145/3579639.
- [2] M. S. I. Shamim, F. A. Bhuiyan, and A. Rahman, "XI Commandments of Kubernetes Security," in *Proc. IEEE SecDev*, Atlanta, GA, 2020, pp. 58–64, doi: 10.1109/SecDev45635.2020.00025.
- [3] J. Elson and A. Cerpa, Internet Content Adaptation Protocol (ICAP), IETF RFC 3507, Apr. 2003. Available: <https://datatracker.ietf.org/doc/html/rfc3507>
- [4] Open Policy Agent Project, OPA: Policy-based Control for Cloud Native Environments, CNCF, 2021. Available: <https://www.openpolicyagent.org>
- [5] Kyverno Project, Kyverno: Kubernetes Native Policy Management, CNCF, 2021. Available: <https://kyverno.io>
- [6] A. Sissodiya et al., "Formal Verification for Preventing Misconfigured Access Policies in Kubernetes Clusters," *IEEE Access*, vol. 13, 2025, doi: 10.1109/ACCESS.2025.3597504.
- [7] Center for Internet Security, CIS Kubernetes Benchmark v1.9, CIS, 2024. Available: <https://www.cisecurity.org/benchmark/kubernetes>
- [8] Payment Card Industry Security Standards Council, PCI DSS v4.0, PCI SSC, 2022. Available: <https://www.pcisecuritystandards.org>
- [9] M. Souppaya, J. Morello, and K. Scarfone, Application Container Security Guide, NIST SP 800-190, NIST, 2017, doi: 10.6028/NIST.SP.800-190.
- [10] F. Minna et al., "Understanding the Security Implications of Kubernetes Networking," *IEEE Security & Privacy*, vol. 19, no. 5, pp. 46–56, 2021.
- [11] C. Cesarano and R. Natella, "KubeFence: Security Hardening of the Kubernetes Attack Surface," in *Proc. 55th IEEE/IFIP DSN*, 2025, doi: 10.1109/DSN64029.2025.00054.
- [12] S. Sultan, I. Ahmad, and T. Dimitriou, "Container Security: Issues, Challenges, and the Road Ahead," *IEEE Access*, vol. 7, pp. 52976–52996, 2019.
- [13] A. Verdet, M. Hamdaqa, L. Da Silva, and F. Khomh, "Exploring Security Practices in Infrastructure as Code," *Empirical Softw. Eng.*, vol. 29, Art. 182, 2024, doi: 10.1007/s10664-024-10610-0.
- [14] L. Rice, Container Security: Fundamental Technology Concepts that Protect Containerized Applications, O'Reilly Media, 2020, ISBN: 978-1492056706.
- [15] B. Burns, B. Grant, D. Oppenheimer, E. Brewer, and J. Wilkes, "Borg, Omega, and Kubernetes," *Commun. ACM*, vol. 59, no. 5, pp. 50–57, 2016, doi: 10.1145/2898442.2898444.
- [16] M. Hausenblas and S. Schimanski, Programming Kubernetes, O'Reilly Media, 2019, ISBN: 978-1492047100.

- [17] J. Dobies and J. Wood, Kubernetes Operators, O'Reilly Media, 2020, ISBN: 978-1492048046.
- [18] PCI Security Standards Council, "Information Supplement: Guidance for Containers and Container Orchestration Tools," v1.0, PCI SSC, Sep. 2022.
- [19] CNCF TAG Security, Cloud Native Security Whitepaper v2, CNCF, 2022. Available: <https://www.cncf.io/reports/cloud-native-security-whitepaper>
- [20] S. Rose, O. Borchert, S. Mitchell, and S. Connelly, Zero Trust Architecture, NIST SP 800-207, NIST, 2020, doi: 10.6028/NIST.SP.800-207.